Generics: Type Erasure

Carsten Gips (HSBI)

TL;DR
Generics existieren eigentlich nur auf Quellcode-Ebene. Nach der Typ-Prüfung etc. entfernt der Compiler alle generischen Typ-Parameter und alle `<...>` (=> "Type-Erasure"), d.h. im Byte-Code stehen nur noch Raw-Typen bzw. die oberen Typ-Schranken der Typ-Parameter, in der Regel `Object`. Zusätzlich baut der Compiler die nötigen Casts ein. Als Anwender merkt man davon nichts, muss das "Type-Erasure" wegen der Auswirkungen aber auf dem Radar haben!

Videos

- VL Generics: Type Erasure

Unterlagen

- PDF-Version

---

1 Typ-Löschung (*Type-Erasure*)

Der Compiler ersetzt nach Prüfung der Typen und ihrer Verwendung alle Typ-Parameter durch

1. deren obere (Typ-)Schranke und
2. passende explizite Cast-Operationen (im Byte-Code).

Die obere Typ-Schranke ist in der Regel der Typ der ersten Bounds-Klausel oder `Object`, wenn keine Einschränkungen formuliert sind.

Bei parametrisierten Typen wie `List<T>` wird der Typ-Parameter entfernt, es entsteht ein sogenannter *Raw*-Typ (`List`, quasi implizit mit `Object` parametrisiert).

=> Ergebnis: Nur **eine** (untypisierte) Klasse! Zur Laufzeit gibt es keine Generics mehr!

Hinweis: In C++ ist man den anderen möglichen Weg gegangen und erzeugt für jede Instantiierung die passende Klasse. Siehe Modul "Systemprogrammierung" :)

Beispiel: Aus dem folgenden harmlosen Code-Fragment:

```
class Studi<T> {
    T myst(T m, T n) { return n; }

    public static void main(String[] args) {
        Studi<Integer> a = new Studi<>();
        int i = a.myst(1, 3);
    }
}
```

---

wird nach der Typ-Löschung durch Compiler (das steht dann quasi im Byte-Code):

```
class Studi {
    Object myst(Object m, Object n) { return n; }

```

```
    public static void main(String[] args) {
        Studi a = new Studi();
        int i = (Integer) a.myst(1, 3);
    }
}
```

Die obere Schranke meist `Object` => `new T()` verboten/sinnfrei (s.u.)!

2 Type-Erasure bei Nutzung von Bounds

vor der Typ-Löschung durch Compiler:

```
class Cps<T extends Number> {
    T myst(T m, T n) {
        return n;
    }

    public static void main(String[] args) {
        Cps<Integer> a = new Cps<>();
        int i = a.myst(1, 3);
    }
}
```

nach der Typ-Löschung durch Compiler:

```
class Cps {
    Number myst(Number m, Number n) {
        return n;
    }

    public static void main(String[] args) {
        Cps a = new Cps();
        int i = (Integer) a.myst(1, 3);
    }
}
```

3 Raw-Types: Ich mag meine Generics “well done” :-)

Raw-Types: Instanziierung ohne Typ-Parameter => `Object`

```
Stack s = new Stack(); // Stack von Object-Objekten
```

- Wegen Abwärtskompatibilität zu früheren Java-Versionen noch erlaubt.
- Nutzung wird nicht empfohlen! (Warum?)

3.1 Anmerkung

Raw-Types darf man zwar selbst im Quellcode verwenden (so wie im Beispiel hier), **sollte** die Verwendung aber vermeiden wegen der Typ-Unsicherheit: Der Compiler sieht im Beispiel nur noch einen Stack für `Object`, d.h. dort dürfen Objekte aller Typen abgelegt werden - es kann keine Typprüfung durch den Compiler stattfinden. Auf einem `Stack<String>` kann der Compiler prüfen, ob dort wirklich nur `String`-Objekte abgelegt werden und ggf. entsprechend Fehler melden.

Etwas anderes ist es, dass der Compiler im Zuge von Type-Erasure selbst Raw-Types in den Byte-Code schreibt. Da hat er vorher bereits die Typsicherheit geprüft und er baut auch die passenden Casts ein.

Das Thema ist eigentlich nur noch aus Kompatibilität zu Java5 oder früher da, weil es dort noch keine Generics gab (wurden erst mit Java6 eingeführt).

4 Folgen der Typ-Löschung: *new*

`new` mit parametrisierten Klassen ist nicht erlaubt!

```
class Fach<T> {
    public T foo() {
        return new T();  // nicht erlaubt!!!
    }
}
```

Grund: Zur Laufzeit keine Klasseninformationen über `T` mehr

Im Code steht `return (CAST)new Object();`. Das neue Object kann man anlegen, aber ein Cast nach irgendeinem anderen Typ ist sinnfrei: Jede Klasse ist ein Untertyp von `Object`, aber eben nicht andersherum. Außerdem fehlt dem Objekt vom Typ `Object` auch sämtliche Information und Verhalten, die der Cast-Typ eigentlich mitbringt …

5 Folgen der Typ-Löschung: *static*

`static` mit generischen Typen ist nicht erlaubt!

```
class Fach<T> {
    static T t;                     // nicht erlaubt!!!
    static Fach<T> c;               // nicht erlaubt!!!
    static void foo(T t) { ... };   // nicht erlaubt!!!
}

Fach<String>  a;
Fach<Integer> b;
```

Grund: Compiler generiert nur eine Klasse! Beide Objekte würden sich die statischen Attribute teilen (Typ zur Laufzeit unklar!).

Hinweis: Generische (statische) Methoden sind erlaubt.

6 Folgen der Typ-Löschung: *instanceof*

`instanceof` mit parametrisierten Klassen ist nicht erlaubt!

```
class Fach<T> {
    void printType(Fach<?> p) {
        if (p instanceof Fach<Number>)
            ...
        else if (p instanceof Fach<String>)
            ...
    }
}
```

Grund: Unsinniger Code nach Typ-Löschung:

```
class Fach {
void printType(Fach p) {
    if (p instanceof Fach)
        ...
    else if (p instanceof Fach)
        ...
    }
}
```

7 Folgen der Typ-Löschung: *.class*

`.class` mit parametrisierten Klassen ist nicht erlaubt!

```
boolean x;
List<String>  a = new ArrayList<String>();
List<Integer> b = new ArrayList<Integer>();

x = (List<String>.class == List<Integer>.class);  // Compiler-Fehler
x = (a.getClass() == b.getClass());               // true
```

Grund: Es gibt nur `List.class` (und kein `List<String>.class` bzw. `List<Integer>.class`)!

8 Wrap-Up

- Generics existieren eigentlich nur auf Quellcode-Ebene
- “Type-Erasure”:
  - Compiler entfernt nach Typ-Prüfungen etc. generische Typ-Parameter etc. => im Byte-Code nur noch Raw-Typen bzw. die oberen Typ-Schranken der Typ-Parameter, in der Regel `Object`
  - Compiler baut passende Casts in Byte-Code ein
  - Transparent für User; Auswirkungen beachten!

9 Zum Nachlesen

- Ullenboom (2021, Kap. 11.2 und 11.6)
- Oracle Corporation (2025)
- Oracle Corporation (2024)
- Bloch (2018)

Lernziele

- k2: Ich verstehe ‘Typ-Löschung’ bei Generics und kann die Auswirkungen erklären

---

Quellen

Bloch, J. 2018. *Effective Java*. 3. Aufl. Addison-Wesley.
Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.
———. 2025. „Learn Java“. 2025. https://dev.java/learn/.
Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** b1291f5 (lecture: improve learning outcomes (Generics Type Erasure), 2025-05-26)